CSCD71 Introduction to High-performance Computing

Parallelizing Short-Time Fourier Transform

Minh Chau Nguyen

December 10, 2024

1 Introduction

The short-time Fourier transform (STFT) provides a time-frequency representation of a signal by dividing it into small overlapping segments, which are then transformed into the frequency domain. It has multiple applications in speech processing, audio analysis, music synthesis, and radar systems. In comparison with the fast Fourier transform, STFT is frequently used to compute Fourier transforms of non-stationary signals, where parameters such as amplitude and frequency experience non-periodic temporal variations, allowing for a more detailed understanding of how the signal evolves.

There are different families of implementations of the STFT, each with their own tradeoffs. The primary goal of this project is to investigate and compare different parallelized implementations of the STFT, particularly in terms of parallel paradigms such as multithreading with OpenMP, multiprocessing with MPI, and hybrid with both.

2 Background

Implementations of the STFT algorithm are mostly categorized into three groups: (1) standard FFT methods and their parallel variants, (2) iterative methods, and (3) feedforward or pruned STFT methods. (1) uses the standard FFT algorithm.

The STFT of a discrete signal $x[n]$ is defined as

$$\mathrm{STFT}_x[n,k] = X[n,k] = \sum_{m=n}^{n+(N-1)} x[m]e^{-j\frac{2\pi k}{N}m}$$

where $k = 0, 1..., N-1$ represents frequency and n represents time.

The definition naturally leads to algorithms belonging to group (1), where N FFTs are computed in parallel. While these algorithms require more hardware, they are straightforward to parallelize as each FFT is calculated independently of one another and only based on the input signal. However, they also incur several redundant computations.

The second approach (2) calculates the STFT for each frequency independently using the iterative equation

$$\begin{aligned}\mathrm{STFT}_x[n,k] =& e^{j\frac{2\pi}{N}k}(\mathrm{STFT}_x[n-1,k] \\ &+ x[n-1+N] - x[n-1])\end{aligned}$$

This group of algorithms have an advantage of hardware cost over the first group, but they also accumulate error at each iteration.

The third group, the feedforward STFT, is based on the radix-2 decimation in time (DIT) FFT, which allows reusing operations of consecutive FFTs. These algorithms have the hardware cost advantage over group (1) and accuracy advantage of group (2), but are much more complicated to parallelize as each step depends on the computations from previous steps, making the algorithm inherently serial.

In this project, we look at a serial and a quasi-parallel implementation of group (3) and compare their performance when parallelizing with OpenMP and MPI with group (1). Group (2) is out of scope for this project.

3 Implementation

3.1 Discrete Fourier transform

To have a baseline level for comparison, an implementation of STFT using discrete Fourier transform (DFT) is provided. As a reminder, the kth element of the DFT of a discrete signal $x[n]$ of size N is given as

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j\frac{2\pi k}{N}n}$$

The STFT algorithm then iterates over the input signal and calls the DFT algorithm passing a given smaller window of the original input signal.

3.1.1 Parallelizing with OpenMP

With OpenMP, the following sections are parallelized:

- Computing each DFT whose input is a subset of the original input signal
- Copying the resulting DFT to a 2D array for output
- Computing each element of the DFT

Relevant functions are

```
rarray<std::complex<double>, 1> fft::dft(
      const rarray<std::complex<double>, 1> &signal,
      size_t vec_begin, size_t vec_size);
rarray<std::complex<double>, 2> fft::stft_dft(
      rarray<std::complex<double>, 1> &vec,
      size_t window_size, size_t window_step);
```

where `dft()` implements the discrete Fourier transform algorithm and `stft_dft()` implements the STFT based on `dft()`.

3.1.2 Parallelizing with MPI

Each MPI process receives an evenly distributed number of DFTs to compute. After computation is completed, each process except rank 0 sends the results to process ranked 0. Process ranked 0 is responsible for aggregating data and outputing the final result.

Relevant functions are

```
rarray<std::complex<double>, 1> fft::dft(
      const rarray<std::complex<double>, 1> &signal,
      size_t vec_begin, size_t vec_size);
rarray<std::complex<double>, 2> fft_mpi::stft_dft(
    rarray<std::complex<double>, 1> &vec,
    size_t window_size, size_t window_step);
```

3.1.3 Parallelizing with OpenMP + MPI

Each MPI process receives an evenly distributed number of DFTs to compute. OpenMP parallelizes the computation of DFTs within one process. After computation is completed, each process except rank 0 sends the results to process ranked 0. Process ranked 0 is responsible for aggregating data and outputing the final result.

Relevant functions are

```
rarray<std::complex<double>, 1> fft::dft(
      const rarray<std::complex<double>, 1> &signal,
      size_t vec_begin, size_t vec_size);
rarray<std::complex<double>, 2> fft_hybrid::stft_dft(
    rarray<std::complex<double>, 1> &vec,
    size_t window_size, size_t window_step)
```

3.2 Fast Fourier transform

The implementation of the FFT-based approach uses a piece of open-source code with some modifications to use `rarray` instead of `std::vector`. This specific modification allows parallelization in later stages as `std::vector` is not thread-safe.

The STFT algorithm then iterates over the input signal and calls the FFT algorithm passing a given smaller window of the original input signal.

3.2.1 Parallelizing with OpenMP

With OpenMP, the following sections are parallelized:

- Computing each FFT whose input is a subset of the original input signal
- Copying the resulting FFT to a 2D array for output
- Computing each element of the FFT

Relevant functions are

```
rarray<std::complex<double>, 1> fft::fft(
      const rarray<std::complex<double>, 1> &signal,
      size_t vec_begin, size_t vec_size);
size_t fft::reverse_bits(size_t val, int width);
bool fft::transform(rarray<std::complex<double>, 1>
↪  &vec);
rarray<std::complex<double>, 2> fft::stft_fft(
      rarray<std::complex<double>, 1> &vec, size_t
      ↪  window_size, size_t window_step);
```

where `fft()` implements the fast Fourier transform algorithm using DIT `transform()` and `reverse_bits()` and `stft_fft()` implements the STFT based on `fft()`.

3.2.2 Parallelizing with MPI

Each MPI process receives an evenly distributed number of FFTs to compute. After computation is completed, each process except rank 0 sends the results to process ranked 0. Process ranked 0 is responsible for aggregating data and outputing the final result.

Relevant functions are

```
rarray<std::complex<double>, 1> fft::fft(
      const rarray<std::complex<double>, 1> &signal,
      size_t vec_begin, size_t vec_size);
size_t fft::reverse_bits(size_t val, int width);
bool fft::transform(rarray<std::complex<double>, 1>
↪  &vec);
rarray<std::complex<double>, 2> fft_mpi::stft_fft(
      rarray<std::complex<double>, 1> &vec, size_t
      ↪  window_size, size_t window_step);
```

3.2.3 Parallelizing with OpeMP + MPI

Each MPI process receives an evenly distributed number of FFTs to compute. OpenMP parallelizes the computation of FFTs within one process. After computation is completed, each process except rank 0 sends the results to process ranked 0. Process ranked 0 is responsible for aggregating data and outputing the final result.

Relevant functions are

```
rarray<std::complex<double>, 1> fft::fft(
      const rarray<std::complex<double>, 1> &signal,
      size_t vec_begin, size_t vec_size);
size_t fft::reverse_bits(size_t val, int width);
```

```
bool fft::transform(rarray<std::complex<double>, 1>
↪  &vec);
rarray<std::complex<double>, 2> fft_hybrid::stft_fft(
      rarray<std::complex<double>, 1> &vec, size_t
      ↪  window_size, size_t window_step);
```

3.3 Serial feedforward STFT

The serial feedforward STFT algorithm uses a circular buffer `b` implemented as a 2D array to store previous computations. The pseudocode for the algorithm is reproduced below

```
for i=1:length(input),
    for s=1:n,
        m = mod(i,N/(2^s));
        for k=0:2:(2^s-1),
            xr = x[s-1][k/2] * TW(s,k,n);
            x[s][k] = b[s][k/2](m) + xr;
            x[s][k+1] = b[s][k/2](m) - xr;
        end
        for k=0:(2^(s-1)-1),
            b[s][k](m) = x[s-1][k];
        end
    end
end
```

Relevant functions are

```
bool transform(rarray<std::complex<double>, 1> &vec,
               rarray<std::complex<double>, 2> &fft,
               rarray<std::complex<double>, 2> &tw);
size_t reverse_bits(size_t val, int width);
rarray<std::complex<double>, 2> fft::stft_ff(
    rarray<std::complex<double>, 1> &vec, size_t
    ↪  window_size, size_t window_step);
```

As discussed above, this implementation is not parallelizable due to its data dependency.

3.4 Quasi-parallel feedforward STFT

The quasi-parallel feedforward STFT is based on the same method as the serial version but uses three 3D arrays to store previous computations, which are fed into a routine that calculates $N/2$ FFTs in parallel (N is the window size of the STFT). The pseudocode for the algorithm is reproduced below

```
# Input: Signal X, window size N, number of stages S_N =
↪  log2(N) + 1
# Output: STFT coefficients for the signal X

# Step 1: Initialization
Create 2D memory buffers M and M' of size (N/2) x (S_N -
↪  1)
Perform FFT on the first N samples of X to initialize M:
  For each stage s = 1 to S_N - 1:
    Store all odd-indexed node-sets in M for stage s

# Step 2: Main Computation Loop
For n = 1 to (length(X) - N):
  For each stage s = 1 to S_N:
    Initialize temporary buffer B for the current batch
    For i = 1 to N/2:
      Fetch prior computations from M[s-1][i]
      If s == 1:
        Collect new signal samples from X and update M'
      Append prior computations and new samples to B
    Compute terminal node-sets for stage s in parallel:
      For each element in the node-set:
        Use invariant equations:
          f_{n,s,|ms|-1,2k}   = f_{n-t,s-1,m,t,k} +
          ↪  twiddle * f_{n,s-1,2|ms|-1,k}
          f_{n,s,|ms|-1,2k+1} = f_{n-t,s-1,m,t,k} -
          ↪  twiddle * f_{n,s-1,2|ms|-1,k}
      Update M and M' with new terminal node-sets

  Output terminal node-sets of stage S_N as STFT
  ↪  coefficients for batch

# Step 3: Buffer Swap
Set M = M' for the next batch of computations
```

3.4.1 Parallelizing with OpenMP

With OpenMP, the following sections are parallelized:

- Preparing input for the first FFT
- Copying result of the first FFT to output variable
- Initializing buffer M
- Computing the next $N/2$ FFTs
- Updating buffer M and M'

Relevant functions are

```
size_t fft::reverse_bits(size_t val, int width);
bool fft::transform(rarray<std::complex<double>, 1>
↪  &vec,
                rarray<std::complex<double>, 2> &fft,
                rarray<std::complex<double>, 2> &tw);
rarray<std::complex<double>, 2> fft::stft_qpff_batch(
      rarray<std::complex<double>, 3> B,
      ↪  rarray<std::complex<double>, 2> tw,
      size_t window_size, size_t s);
rarray<std::complex<double>, 2> fft::stft_qpff(
      rarray<std::complex<double>, 1> &vec, size_t
      ↪  window_size, size_t window_step);
```

3.4.2 Parallelizing with MPI

Process ranked 0 broadcasts the input buffer B. Each MPI process receives an evenly distributed number of FFTs among $N/2$ FFTs to compute. After computation is completed, each process except rank 0 sends the results to process ranked 0. Process ranked 0 is responsible for aggregating data and updating the final result.

Relevant functions are

```
size_t fft::reverse_bits(size_t val, int width);
bool fft::transform(rarray<std::complex<double>, 1>
↪  &vec,
                rarray<std::complex<double>, 2> &fft,
                rarray<std::complex<double>, 2> &tw);
rarray<std::complex<double>, 2>
↪  fft_mpi::stft_qpff_batch(
      rarray<std::complex<double>, 3> B,
      ↪  rarray<std::complex<double>, 2> tw,
      size_t window_size, size_t s);
rarray<std::complex<double>, 2> fft_mpi::stft_qpff(
      rarray<std::complex<double>, 1> &vec, size_t
      ↪  window_size, size_t window_step);
```

3.4.3 Parallelizing with OpenMP + MPI

Process ranked 0 broadcasts the input buffer B. Each MPI process receives an evenly distributed number of FFTs among $N/2$ FFTs to compute. After computation is completed, each process except rank 0 sends the results to process ranked 0. Process ranked 0 is responsible for aggregating data and updating the final result.

In addition, the same sections are parallelized with OpenMP as in part (a).

Relevant functions are

```
size_t fft::reverse_bits(size_t val, int width);
bool fft::transform(rarray<std::complex<double>, 1>
↪  &vec,
                    rarray<std::complex<double>, 2> &fft,
                    rarray<std::complex<double>, 2> &tw);
rarray<std::complex<double>, 2>
↪  fft_hybrid::stft_qpff_batch(
      rarray<std::complex<double>, 3> B,
      ↪  rarray<std::complex<double>, 2> tw,
      size_t window_size, size_t s);
rarray<std::complex<double>, 2> fft_hybrid::stft_qpff(
      rarray<std::complex<double>, 1> &vec, size_t
      ↪  window_size, size_t window_step);
```

4 Analysis

4.1 Discrete Fourier transform

4.1.1 Profiling

Profiling is done with `num-samples=1024` and `window-size=512`.

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call
 ↪  name
100.02      0.88     0.88      513     1.72     1.72
↪  fft::dft(ra::rarray<std::complex<double>, 1> const&,
↪  unsigned long, unsigned long)
  0.00      0.88     0.00        1     0.00     0.00
  ↪  _GLOBAL__sub_I__[...]
  0.00      0.88     0.00        1     0.00     0.00
  ↪  ra::rarray<std::complex<double>, 2>::rarray<2,
  ↪  void>(long, long)
  0.00      0.88     0.00        1     0.00   880.16
  ↪  fft::stft_dft(ra::rarray<std::complex<double>,
  ↪  1>&, unsigned long, unsigned long)
```

As expected, most of the time is spent on computing individual DFTs.

4.1.2 Parallelizing with OpenMP

(a) Strong scaling

Run strong scaling analysis with the following arguments `sbatch jobscript-omp.sh ./stft -a dft -p omp -s 16384 256`.

```
1 114.35 1
2 57.47 1.9897337714303114668
3 38.28 2.9871995820271682 3406
4 29.06 3.9349621472814865 7949
5 23.89 4.7865215571368773 5454
6 20.19 5.6636948984645864 2892
7 18.43 6.2045577862181226 2615
8 16.28 7.0239557739557739 5577
9 14.53 7.8699242945629731 5898
10 13.28 8.6106927710843373 4939
11 11.95 9.5690376569037656 9037
12 11.09 10.311091073038773 66997
13 10.55 10.838862559241706 16113
14 9.82 11.644602851323828 92057
15 9.23 12.388949079089924 16034
16 8.92 12.819506726457399 10313
32 8.98 12.733853006681514 47661
48 8.77 13.038768529076396 80729
64 8.69 13.158803222094361 33486
```

Based on Figure 1, from 1 to 16 threads, the speedup is almost linear as we may expect from a close to embarrassingly parallel problem such as this one. From 32 to 64 threads (hyperthreading), not much improvement in performance is observed.

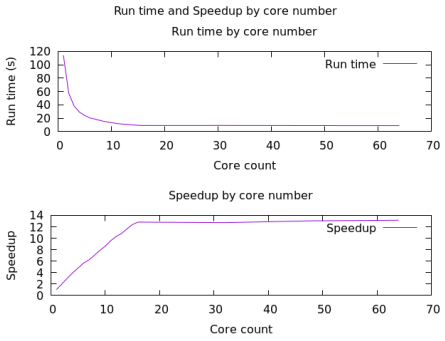

Figure 1: Runtime and speedup of DFT-based STFT with OpenMP - Strong scaling

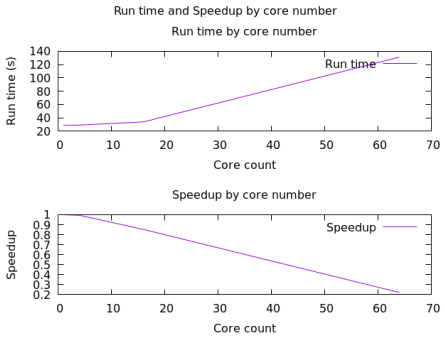

Figure 2: Runtime and speedup of DFT-based STFT with OpenMP - Weak scaling

(b) Weak scaling

Time complexity of DFT is $\mathcal{O}(N^2)$, so to run weak scaling analysis, we increase the number of threads exponentially compared to the window size. We keep the size of the input signal constant. Below is the result of weak scaling analysis with input signal size 16384, window size $128 \times \{1, 2, 4, 8\}$, and thread count $\{1, 4, 16, 64\}$.

```
1 28.83 1
4 29.00 .99413793103448275862
16 33.81 .85270629991126885536
64 131.23 .21969061952297492951
```

Based on Figure 2, runtime is very close to constant from 1 to 16 threads, as what we would expect from a problem close to embarrassingly parallel. At 64 threads, runtime surges, which could potentially be explained by hyperthreading.

4.1.3 Parallelizing with MPI

(a) Strong scaling

Run strong scaling analysis with the following arguments `sbatch jobscript-mpi.sh`

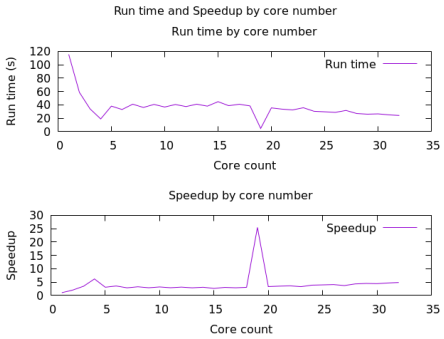

Figure 3: Runtime and speedup of DFT-based STFT with MPI - Strong scaling

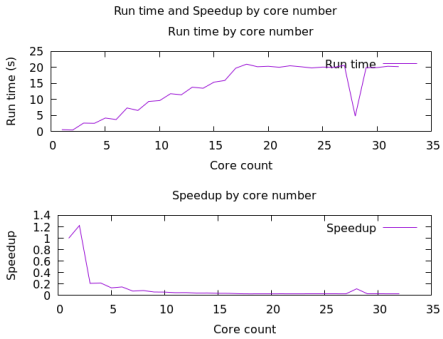

Figure 4: Runtime and speedup of DFT-based STFT with MPI - Weak scaling

`./stft -a dft -p mpi -s 16384 256`. 32 cores were used to observe both on-node and off-node communication.

```
1 115.61 1
2 58.65 1.9711849957374254049 4
3 33.86 3.4143532191376255168 3
4 18.80 6.1494608510638297872 5
5 38.03 3.0399644596371285826 6
6 32.95 3.5086494688922610015 1
7 40.94 2.8238886174890083048 3
8 36.02 3.2096057745696835091 6
9 40.59 2.8482384823848238482 3
10 36.67 3.1527133896918461958 0
11 40.47 2.8566839634297010130 9
12 37.36 3.0944860813704496788 0
13 40.94 2.8238886174890083048 3
14 37.97 3.0447721885699236239 1
15 44.69 2.5869321995972253300 5
16 38.70 2.9873385012919896640 8
17 40.85 2.8301101591187270501 8
18 38.33 3.0161753195930080876 5
19 4.54 25.464757709251101321 58
20 35.37 3.2685891998869098105 7
21 33.50 3.4510447761194029850 7
22 32.33 3.5759356634704608722 5
23 35.61 3.2465599550688008986 2
24 30.31 3.8142527218739689871 3
25 29.48 3.9216417910447761194 0
26 28.65 4.0352530541012216404 8
27 31.53 3.6666666666666666666 6
28 26.95 4.2897959183673469387 7
29 25.93 4.4585422290782876976 4
30 26.39 4.3808260704812428950 3
31 24.98 4.6281024819855884707 7
32 23.96 4.8251252086811352253 7
```

Based on Figure 4, MPI improvement is noticeable from 1 to 4 cores, but from 5 cores onwards there is no significant change.

(b) Weak scaling

Weak scaling analysis is done 32 cores for both on-node and off-node behavior. The window size is kept constant and the size of the input signal is increased at the same rate as the number of cores. This is because parallelization is done on the number of individuals DFTs, which is determined by the size of the input signal.

```
1 0.55 1
2 0.45 1.2222222222222222222
3 2.62 .20992366412213740458
4 2.55 .21568627450980392156
5 4.20 .13095238095238095238
6 3.73 .14745308310991957104
7 7.35 .07482993197278911564
8 6.57 .08371385083713850837
9 9.39 .05857294994675186368
10 9.65 .05699481865284974093
11 11.80 .04661016949152542372
12 11.47 .04795117698343504795
13 13.84 .03973988439306358381
14 13.51 .04071058475203552923
15 15.42 .03566796368352788586
16 15.94 .03450439146800501882
17 19.78 .02780586450960566228
18 21.04 .02614068441064638783
19 20.24 .02717391304347826086
20 20.38 .02698724239450441609
21 20.07 .02740408570004982561
22 20.54 .02677702044790652385
23 20.23 .02718734552644587246
24 19.86 .02769385699899295065
25 20.13 .02732240437158469945
26 20.04 .02744510978043912175
27 20.66 .02662149080348499515
28 4.80 .11458333333333333333
29 19.95 .02756892230576441102
30 19.97 .02754131196795192789
31 20.38 .02698724239450441609
32 20.27 .02713369511593487913
```

Based on Figure 4, it appears that MPI parallelization does not boost performance of DFT as runtime steadily increases with the amount of work per core and the number of cores.

4.1.4 Parallelizing with OpenMP + MPI

Run scaling analysis job for 1 to 8 processes, each process with 1 to 8 threads with `num-samples=10000` and `window-size=256`.

4.2 Fast Fourier transform

4.2.1 Profiling

Profiling is done with `num-samples=1024` and `window-size=512`.

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls  us/call  us/call
 ↪  name
100.02      0.01     0.01      384    26.05    26.05
↪  fft::transform(ra::rarray<std::complex<double>, 1>&)
  0.00      0.01     0.00        1     0.00     0.00
  ↪  _GLOBAL__sub_I__[...]
```

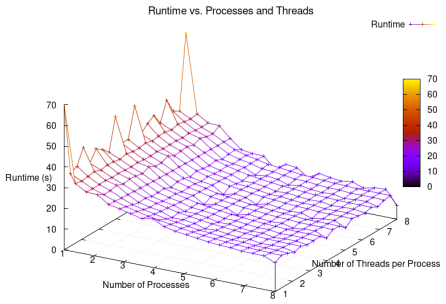

Figure 5: Runtime of DFT-based STFT with OpenMP + MPI

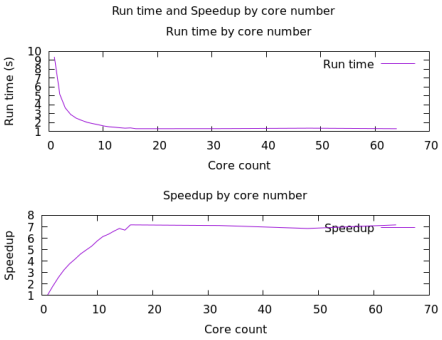

Figure 6: Runtime and speedup of FFT-based STFT with OpenMP - Strong scaling

```
0.00      0.01      0.00         1      0.00      0.00
↪  ra::rarray<std::complex<double>, 2>::rarray<2,
↪  void>(long, long)
0.00      0.01      0.00         1      0.00      0.00
↪  fft::stft_fft(ra::rarray<std::complex<double>,
↪  1>&, unsigned long, unsigned long)
```

As one may expect, the majority of runtime is spent on computing individual FFTs.

4.2.2 Parallelizing with OpenMP

(a) Strong scaling

Run strong scaling analysis with the following arguments `sbatch jobscript-omp.sh ./stft -a fft -p omp -s 131072 1024`.

```
1 9.39 1
2 5.17 1.8162475822050290 1353
3 3.65 2.5726027397260273 9726
4 2.91 3.2268041237113402 0618
5 2.50 3.7560000000000000 0000
6 2.25 4.1733333333333333 3333
7 2.03 4.6256157635467980 2955
8 1.89 4.9682539682539682 5396
9 1.77 5.3050847457627118 6440
10 1.63 5.7607361963190184 0490
11 1.53 6.1372549019607843 1372
12 1.48 6.3445945945945945 9459
13 1.42 6.6126760563380281 6901
14 1.37 6.8540145985401459 8540
15 1.40 6.7071428571428571 4285
16 1.31 7.1679389312977099 2366
32 1.32 7.1136363636363636 3636
48 1.37 6.8540145985401459 8540
64 1.31 7.1679389312977099 2366
```

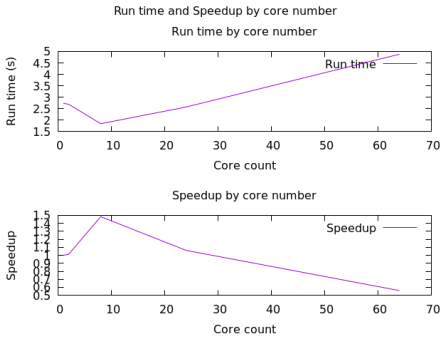

Figure 7: Runtime and speedup of FFT-based STFT with OpenMP - Weak scaling

Based on Figure 6, from 1 to 16 threads, the speedup is almost linear as we may expect from a close to embarrassingly parallel problem such as this one. From 32 to 64 threads (hyperthreading), not much improvement in performance is observed.

(b) Weak scaling

Time complexity of FFT is $\mathcal{O}(N \log N)$, so to run weak scaling analysis, we increase the number of threads $N \log N$ times compared to the window size. We keep the size of the input signal constant. Below is the result of weak scaling analysis with input signal size 131072, window size 256 $\times\{1, 2, 4, 8, 16\}$, and thread count $\{1, 2, 8, 24, 64\}$.

```
1 2.73 1
2 2.69 1.0148698884758364 3122
8 1.84 1.4836956521739130 4347
24 2.57 1.0622568093385214 0077
64 4.88 .55942622950819672131
```

See Figure 7 for a plot of runtime and speedup by thread count.

4.2.3 Parallelizing with MPI

(a) Strong scaling

Run strong scaling analysis with the following arguments `sbatch jobscript-mpi.sh ./stft -a dft -p mpi -s 16384 256`. 32 cores were used to observe both on-node and off-node communication.

```
1 0.84 1
2 0.64 1.3125000000000000 0000
3 2.92 .28767123287671232876
4 0.74 1.1351351351351351 3513
5 30.65 .02740619902120717781
6 18.57 .04523424878836833602
7 39.84 .02108433734939759036
8 49.76 .01688102893890675241
9 84.40 .00995260663507109004
10 83.16 .01010101010101010101
11 92.06 .00912448403215294373
12 84.88 .00989632422243166823
13 92.10 .00912052117263843648
14 85.94 .00977426111240400279
15 92.54 .00907715582450832072
16 87.46 .00960439057855019437
```

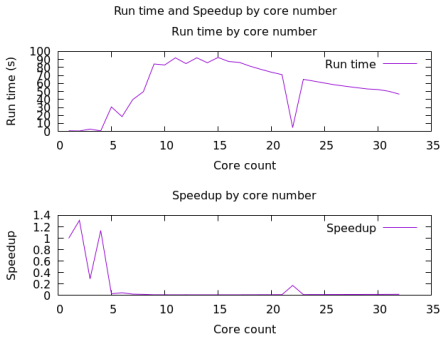

Figure 8: Runtime and speedup of FFT-based STFT with MPI - Strong scaling

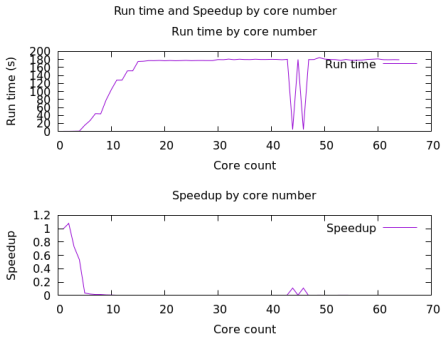

Figure 9: Runtime and speedup of FFT-based STFT with MPI - Weak scaling

```
17 86.36 .00972672533580361278
18 81.66 .01028655400440852314
19 77.74 .01080524826344224337
20 74.11 .01133450276615841316
21 71.07 .01181933305192064162
22 4.83 .17391304347826086956
23 65.19 .01288541187298665439
24 62.90 .01335453100158982511
25 60.52 .01387970918704560475
26 58.22 .01442803160425970456
27 56.47 .01487515494953072427
28 54.77 .01533686324630272046
29 53.18 .01579541180895073335
30 52.42 .01602441816100724914
31 50.30 .01669980119284294234
32 46.64 .01801029159519725557
```

(b) Weak scaling

Weak scaling analysis is done 64 cores for both on-node and off-node behavior. The window size is kept constant and the size of the input signal is increased at the same rate as the number of cores. This is because parallelization is done on the number of individuals FFTs, which is determined by the size of the input signal.

```
1 0.54 1
2 0.50 1.08000000000000000000
3 0.74 .72972972972972972972
4 1.01 .53465346534653465346
5 15.52 .03479381443298969072
6 27.29 .01978746793697325027
7 44.57 .01211577294144043078
8 44.05 .01225879682179341657
9 78.43 .00688512048960856814
10 104.99 .00514334698542718354
11 128.37 .00420659032484225286
12 128.41 .00420527996261973366
13 151.55 .00356318046849224678
14 151.64 .00356106568187813241
15 175.05 .00308483290488431876
16 175.61 .00307499572917259837
17 177.20 .00304740406320541760
18 177.07 .00304964138476308804
19 177.57 .00304105423213380638
20 176.81 .00305412589785645608
21 177.42 .00304362529590801487
22 176.85 .00305343511450381679
23 177.21 .00304723209751142712
24 177.72 .00303848750844024307
25 177.05 .00304998587969500141
26 177.35 .00304482661404003383
27 177.39 .00304414003044140030
28 177.26 .00304637256008123660
29 177.32 .00304534175501917437
30 179.84 .00302066903914590747
31 179.57 .00300718382803363590
```

```
32 181.07 .00298227204948362511
33 179.60 .00300668151447661469
34 180.81 .00298656047784967645
35 180.02 .00299966670369958893
36 179.92 .00300133392618941751
37 180.79 .00298689086785773549
38 180.15 .00299750208159866777
39 180.17 .00299716934006771382
40 180.50 .00299168975069252077
41 180.32 .00299467613132209405
42 179.46 .00300902708124373119
43 180.51 .00299152401529001163
44 4.94 .10931174089068825910
45 180.01 .00299983334259207821
46 4.95 .10909090909090909090
47 179.80 .00300333704115684093
48 180.02 .00299966670369958893
49 184.69 .00292381829010774811
50 181.06 .00298243676129459847
51 180.13 .00299783489701881974
52 179.76 .00300400534045393858
53 177.76 .00303780378037803780
54 180.20 .00299667036625971143
55 177.96 .00303438975050573162
56 178.27 .00302911314298535928
57 178.48 .00302554908112953832
58 180.19 .00299683667240135412
59 180.78 .00298705609027547295
60 181.51 .00297504269737204561
61 179.62 .00300634673198975615
62 179.41 .00300986567080987681
63 179.73 .00300450676014021031
64 179.44 .00300936246098974587
```

Based on Figure 4, it appears that MPI parallelization does not boost performance of FFT as runtime steadily increases with the amount of work per core and the number of cores.

4.2.4 Parallelizing with OpenMP + MPI

Run scaling analysis job for 1 to 8 processes, each process with 1 to 8 threads with `num-samples=1000000` and `window-size=1024`.

4.3 Serial feedforward STFT

4.3.1 Profiling

Profiling is done with `num-samples=131072` and `window-size=1024`.

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls   s/call   s/call
 ↪  name
```

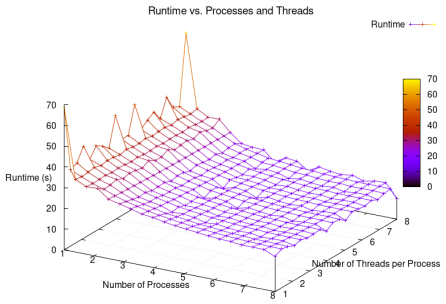

Figure 10: Runtime of FFT-based STFT with OpenMP + MPI

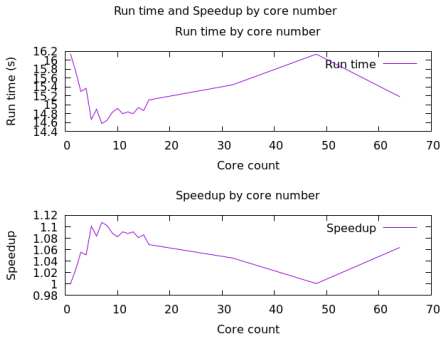

Figure 11: Runtime and speedup of quasi-parallel feedforward STFT with OpenMP - Strong scaling

```
 81.77      2.42     2.42        1     2.42     2.96
 ↪  fft::stft_ff(ra::rarray<std::complex<double>, 1>&,
 ↪  unsigned long, unsigned long)
 18.25      2.96     0.54        4     0.14     0.14
 ↪  ra::rarray<std::complex<double>, 2>::rarray<2,
 ↪  void>(long, long)
  0.00      2.96     0.00        1     0.00     0.00
  ↪  _GLOBAL__sub_I__[...]
  0.00      2.96     0.00        1     0.00     0.00
  ↪  fft::transform(ra::rarray<std::complex<double>,
  ↪  1>&, ra::rarray<std::complex<double>, 2>&,
  ↪  ra::rarray<std::complex<double>, 2>&)
```

The majority of runtime is spent in the serial fraction of the function `stft_ff()` as one may expect. Additionally, the time to allocate arrays for buffers and output also contribute significantly to the total runtime.

4.3.2 Comparison

```
$ export OMP_NUM_THREADS=1
$ time ./stft -a ff -p omp -s 131072 1024

real        0m3.005s
user        0m2.511s
sys            0m0.441s
$ time ./stft -a fft -p omp -s 131072 1024

real        0m9.240s
user        0m8.459s
sys            0m0.776s
$ time ./stft -a qpff -p omp -s 131072 1024

real        0m16.836s
user        0m13.070s
sys            0m3.703s
```

The serial feedforward STFT outperforms FFT-based STFT and quasi-parallel feedforward STFT on single core, single thread.

4.4 Quasi-parallel feedforward STFT

4.4.1 Profiling

Profiling is done with `num-samples=1024` and `window-size=512`.

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call
 ↪  name
 50.01      0.01     0.01        2     5.00     5.00
 ↪  ra::rarray<std::complex<double>, 3>::copy() const
```

```
 50.01      0.02     0.01        1    10.00    20.00
 ↪  fft::stft_qpff(ra::rarray<std::complex<double>,
 ↪  1>&, unsigned long, unsigned long)
  0.00      0.02     0.00       23     0.00     0.00
  ↪  ra::rarray<std::complex<double>, 2>::rarray<2,
  ↪  void>(long, long)
 0.00      0.02     0.00        5     0.00     0.00
  ↪  ra::detail::shared_shape<std::complex<double>,
  ↪  3>::shared_shape(std::array<long, 3ul> const&,
  ↪  std::complex<double>*)
 0.00      0.02     0.00        1     0.00     0.00
  ↪  _GLOBAL__sub_I__[...]
 0.00      0.02     0.00        1     0.00     0.00
  ↪  fft::transform(ra::rarray<std::complex<double>,
  ↪  1>&, ra::rarray<std::complex<double>, 2>&,
  ↪  ra::rarray<std::complex<double>, 2>&)
```

Half of the time is spent on copying buffers to prepare for the next iteration while the other half is the actual computation time.

4.4.2 Parallelizing with OpenMP

(a) Strong scaling

Run strong scaling analysis with the following arguments `sbatch jobscript-omp.sh ./stft -a qpff -p omp -s 131072 1024`.

```
1 16.15 1
2 15.76 1.0247461928934010152
3 15.30 1.0555555555555555555
4 15.37 1.0507482108002602472 3
5 14.67 1.1008861622358554873 8
6 14.90 1.0838926174496644295 3
7 14.58 1.1076817558299039780 5
8 14.65 1.1023890784982935153 5
9 14.83 1.0890087660148347943 3
10 14.92 1.0824396782841823056 3
11 14.80 1.0912162162162162162
12 14.84 1.0882749326145552560 6
13 14.80 1.0912162162162162162
14 14.94 1.0809906291834002677 3
15 14.87 1.0860793544048419636 8
16 15.11 1.0688285903375248180 0
32 15.45 1.0453074433656957928 8
48 16.14 1.0006195786864931846 3
64 15.18 1.0638998682476943346 5
```

Based on Figure 11, the improvement in performance with increasing thread count is small and inconsistent. At around 6 threads, runtime seems to plateau with no further significant improvement.

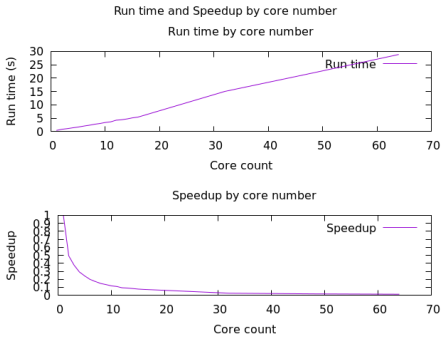

Figure 12: Runtime and speedup of quasi-parallel feedforward STFT with OpenMP - Weak scaling

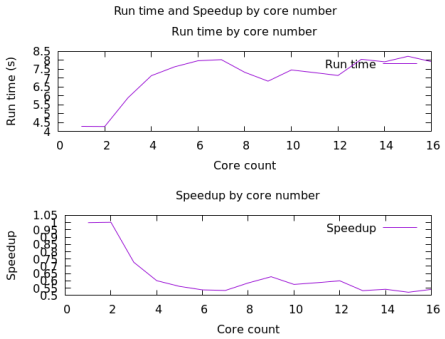

Figure 13: Runtime and speedup of quasi-parallel feedforward STFT with MPI

(b) Weak scaling

Run weak scaling analysis with increasing input signal size as the algorithm parallelizes based on the number of discrete input signals.

```
1 0.40 1
2 0.81 .49382716049382716049
3 1.06 .37735849056603773584
4 1.37 .29197080291970802919
5 1.65 .24242424242424242424
6 2.01 .19900497512437810945
7 2.32 .17241379310344827586
8 2.70 .14814814814814814814
9 2.98 .13422818791946308724
10 3.37 .11869436201780415430
11 3.61 .11080332409972299168
12 4.23 .09456264775413711583
13 4.40 .09090909090909090909
14 4.70 .08510638297872340425
15 5.11 .07827788649706457925
16 5.38 .07434944237918215613
32 15.03 .02661343978709248170
48 21.87 .01828989483310470964
64 28.88 .01385041551246537396
```

See Figure 12 for a plot of runtime and speedup by thread count.

4.4.3 Parallelizing with MPI

```
1 4.29 1
2 4.28 1.00233644859813084112
3 5.90 .72711864406779661016
4 7.14 .60084033613445378151
5 7.64 .56151832460732984293
6 7.98 .53759398496240601503
7 8.04 .53358208955223880597
8 7.33 .58526603001364256480
9 6.83 .62811127379209370424
10 7.46 .57506702412868632707
11 7.31 .58686730506155950752
12 7.15 .60000000000000000000
13 8.06 .53225806451612903225
14 7.92 .54166666666666666666
15 8.23 .52126366950182260024
16 7.93 .54098360655737704918
```

4.4.4 Parallelizing with OpenMP + MPI

Run scaling analysis job for 1 to 8 processes, each process with 1 to 8 threads with `num-samples=1024` and `window-size=256`.

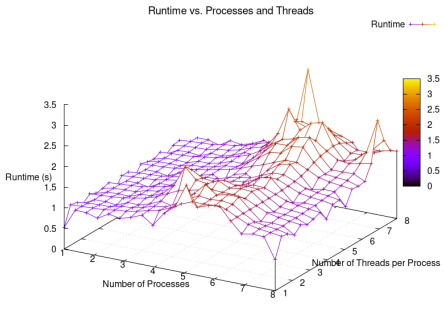

Figure 14: Runtime of quasi-parallel feedforward STFT with OpenMP + MPI

5 Discussion

Further improvements to be considered:

- Revise weak scaling problem size: Currently, weak scaling is done on either the size of the input signal or the window size based on the parallelization of the algorithm. We may benefit from a more comprehensive scaling with both parameters.
- Reduce overhead of working with `rarray`: `std::ostream& ra::detail::text_output<std::complex<double>, 2>(std::ostream&, ra::rarray<std::complex<double>, 2> const&)` is a significant contributor to the overall runtime of MPI parallelization. We may see better performance on MPI if `rarray` output can be managed.
- Review nested loops: The feedforward algorithms use deeply nested loops for its main section. Data are accessed out of order and may result in cache thrashing. A potential improvement is to ensure that data are well structured for cachelines and avoid wasting time on out of sequence accessing.
- More efficient memory management: During analysis, some outliers can be observed

due to program failure. Most of the errors recorded are related to insufficient memory. We may want to investigate further this issue and design a more efficient version in terms of memory.

Conclusions

Overall, there is a significant performance improvement for group (1) of algorithms based on DFT and FFT with OpenMP. This is quite unexpected as this group of algorithms are particularly natural to parallelize. However, when parallelizing with MPI, the performance seems to suffer. This could potentially be accounted for by a communication overhead, especially as the number of processes increases the root process requires more time to gather outputs from other processes.

Even though not parallelizable, the serial feedforward STFT outperforms the FFT-based STFT on single core, single thread. This boost in performance results from reducing the number of computations. However, the quasi-parallel feedforward STFT has worse performance on single core, single thread than both the serial feedforward and the FFT-based STFT, possibly due to the overhead of creating and maintaining buffers.

The quasi-parallel feedforward STFT does not improve much in terms of performance when parallelizing with either OpenMP, MPI, or a hybrid of both. Profiling the code shows that the majority of time is spent of maintaining the buffers, which scale with the size of the problem.

References

[1] Alfonso B. Labao, Rodolfo C. Camaclang, and Jaime D.L. Caro. 2019. Staggered parallel short-time Fourier transform. Digit. Signal Process. 93, C (Oct 2019), 70–86. https://doi.org/10.1016/j.dsp.2019.07.003

[2] M. Garrido, "The Feedforward Short-Time Fourier Transform," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 63, no. 9, pp. 868-872, Sept. 2016, doi: 10.1109/TCSII.2016.2534838. keywords: Feedforward neural networks;Hardware;Adders;Time-frequency analysis;Iterative methods;Mathematical model;Feedforward;fast Fourier transform (FFT);pipelined architecture;short-time Fourier transform (STFT),